

COL100: Introduction to Computer Science

Lab 2: Operators

January 13, 2025

1 Operators

1.1 Objects and Types

Objects are the core things that Python programs manipulate. Each object has a **type** that defines what programs can do with that object. **Types** are either scalar or non-scalar. **Scalar** objects are indivisible. Think of them as the atoms of the language. **Non-scalar** objects, for example strings, have internal structure. Many **types of objects** can be denoted by literals in the text of a program. For example, the text **2** is a literal representing an integer and the text **3.0** is a literal representing a floating point number.

Python has four types of scalar objects:

- *int* is used to represent integers.
- *float* is used to represent real numbers.
- *bool* is used to represent the Boolean values **True** and **False**.
- *None* is a type with a single value.

1.2 Variables

Variables provide a way to associate names with objects. In Python, a **variable** is a named location in memory used to store data. You can think of it as a container that holds a value.

Key points about Python variables:

- Creating a variable: You create a variable simply by assigning a value to it:

```
1 x = 25          # integer
2 y = 5.0         # float
```

- Dynamic typing: Python is dynamically typed, which means the type of a variable is determined at runtime. You can assign different types of values to the same variable:

```
1 x = 10          # x is of type int
2 x = 5.0         # x is now of type float
```

- Naming rules: Variable names must follow certain rules:

- They can contain letters (a-z, A-Z), digits (0-9), and underscores (_).
- They must start with a letter or an underscore.
- They are case-sensitive (e.g., myVar and myvar are different variables).
- Use descriptive names to make your code easier to understand.

Accessing a variable: You can access the value stored in a variable by using its name:

```
1 x = 5.0
2 print(x)    # Output: 5.0
```

Modifying a variable: You can change the value stored in a variable by assigning a new value to it:

```
1 x = 20    # x now holds the value 20
```

1.3 Variable Types

Python has several basic variable types. In this lab we will focus on Integers.

In Python, you don't explicitly declare the type of a variable. You simply assign a value to a variable, and Python infers the type. Here's how you create an integer variable in Python:

```
1 x = 10    # x is an integer variable
```

Integers represent whole numbers.

Keypoints:

- No Declaration: You don't need to use keywords like `int` to declare the variable type as an integer.
- Type Inference: Python automatically determines that `x` is an integer based on the assigned value.
- Checking the Type: You can use the `type()` function to confirm the type of a variable:

```
1 print(type(x))    # Output: <class 'int'>
```

1.4 Input and Output in Python

Python makes it easy to handle input and output (I/O). You can show messages to the user and take input from them. We use functions to achieve this. A function is like a recipe or set of instructions that the computer follows to do a specific task. Once written functions can be reused. Parameters are like the ingredients in the recipe. In the same way, when you execute a function, you give it some values (parameters) to work with. This section explains how to display text on the screen and get data from the user.

1.4.1 Output in Python

To print output to the screen, you use the `print()` function. Here's an example:

```
1 print("Hello, world!")
```

In the previous lab, we used the above statement to output `Hello, world!` to the console. The `print()` function automatically adds a newline after the message unless specified otherwise. You can also print multiple variables in one call using commas. Let's try that with a few numerical values.

```
1 # Output in Python
2 x = 5.0
3 y = 7
4 z = 1 + 2j
5 print(x, y, z)
6 print(x,y,z, sep="--")
7 print(x,y,z, end="**")
8 print(x)
```

```
1 5.0 7 (1+2j)
2 5.0--7--(1+2j)
3 5.0 7 (1+2j)**5.0
```

By adding the `sep` and `end` parameters we modify how the print function behaves. In the first line we show that multiple variables can be print using commas, with the default space separator. In the second command, we see the space separator replaced with `--`, and in the third we see the default new line which we saw in execution of the first two print calls be relaced with `**`.

You can print variables by including them inside the `print()` function. Python also allows formatted strings to make this easier. Here's an example:

```
1 num = 10
2 print("The number is", num) # Output: The number is 10
3
4 # Printing a variable using formatted string
5 print(f"The number is {num}") # Output: The number is 10
```

In this example, `f"The number is {num}"` is an f-string that inserts the value of `num` into the string.

```
1 name = "Alice"
2 age = 25
3 print("Name:", name, "Age:", age)
```

Try: Use the `print()` function to output multiple variables in a formatted way.

1.4.2 Input in Python

To get input from the user, Python uses the `input()` function, which reads data as a string. Here's an example:

```

1 # Input in Python
2 name = input("Enter your name: ")
3 print(f"Hello, {name}!")

```

In this example, `input()` displays the prompt message and waits for the user to type something. Whatever the user enters is stored in the variable `name`.

Since `input()` returns a string, if you need a specific data type like an integer or a float, you can convert the input using typecasting. Here's how to read an integer:

```

1 # Input with typecasting
2 age = int(input("Enter your age: "))
3 print(f"You are {age} years old.")

```

In this example, `int()` converts the string input into an integer.

Try: Read two numbers from the user and print their sum.

1.4.3 Summary

- Use `print()` to display output.
- Use `input()` to read data from the user (returns a string).
- Typecast the input if you need a different data type (e.g., `int()`, `float()`).

1.5 Operators in Python

Operators in Python are special symbols that perform operations on variables and values. They allow you to manipulate data and execute various computations. This section introduces some commonly used operators in Python.

1.5.1 Arithmetic Operators

Arithmetic operators in Python are used to perform mathematical operations on operands. Here are some basic arithmetic operators:

- `+` Addition: Adds two operands.
- `-` Subtraction: Subtracts the second operand from the first.
- `*` Multiplication: Multiplies two operands.
- `/` Division: Divides the first operand by the second (always returns a float).
- `//` Floor Division: Divides the first operand by the second and rounds down to the nearest integer.
- `%` Modulus: Returns the remainder of the division of the first operand by the second.
- `**` Exponentiation: Raises the first operand to the power of the second.

1.5.2 Example Program: Arithmetic Operations

Here's an example Python program that takes two numbers as input and performs addition, subtraction, multiplication, division, floor division, modulus, and exponentiation:

```
1 # Arithmetic operations in Python
2 num1 = int(input("Enter first number: "))
3 num2 = int(input("Enter second number: "))
4
5 print(f"Addition: {num1 + num2}")
6 print(f"Subtraction: {num1 - num2}")
7 print(f"Multiplication: {num1 * num2}")
8 print(f"Division: {num1 / num2}")
9 print(f"Floor Division: {num1 // num2}")
10 print(f"Modulus: {num1 % num2}")
11 print(f"Exponentiation: {num1 ** num2}")
```

In this program: - / gives the floating-point result of division. - // performs floor division, which gives an integer result by rounding down. - ** is used for exponentiation. **Try:** What happens when you input zero as the second number in division or modulus operations?

1.5.3 Summary

- Python supports basic arithmetic operators like addition (+), subtraction (-), multiplication (*), and division (/).
- // performs floor division, % gives the remainder, and ** performs exponentiation.
- Python handles division more flexibly, returning floating-point results by default.

1.6 Some Other Variable Types

As you have already learned about integers, let's explore other variable types supported by Python.

1.6.1 Floating-Point Numbers

Floating-point numbers represent real numbers, including decimals. In Python, they are represented as float type.

```
1 pi = 3.14159      # floating-point number
2 e = 2.71828       # another floating-point number
```

1.6.2 Boolean

Boolean variables hold True or False values in Python.

```
1 is_passed = True
```

1.6.3 Constants

Python does not have a specific syntax for constants. However, by convention, variables that should not change are written in uppercase.

```
1 MAX_SCORE = 100 # constant by convention
```

Try: Take the input for a float value and display it. Do the same for a bool value. You can use `input()` for both.

2 Submission Problem

2.1 Instructions

- Complete the code of following problem in **complex_mult.py** file on Anaconda.
- Run and test your code for some sample test cases.
- Submit the file in the assignment titled “Lab 2” on Gradescope.
- Autograder on Gradescope will evaluate the submitted code on some hidden test cases.

2.2 Complex Number Multiplication

Write a Python program that calculates the product of two complex numbers. A complex number is represented in the form “ $a + bi$ ”, where “ a ” is the real part and “ b ” is the imaginary part.

2.2.1 Program Requirements

- Your program should take the real and imaginary parts of the two complex numbers as input from the user. Consider “int” values as inputs.
- It should calculate the product of the two complex numbers.
- It should display the real and imaginary parts of the product complex number on two different lines.

Here is a sample input and output of the program.

2.2.2 Input

```
1 2
2 3
3 4
4 -5
```

2.2.3 Output

```
1 23
2 2
```

2.2.4 Explanation

Here the two input complex numbers are: $2 + 3i$ and $4 - 5i$. After running your code on the given input, it should generate the product of the input complex numbers, i.e., $23 + 2i$.

3 Practice Problems

These practice problems will not be graded and are for your practice. You can try to solve these questions in the lab and get your doubts cleared by the teaching assistants.

3.1 Temperature Scale Conversion

Write a Python program that converts a temperature in degrees Celsius to degrees Fahrenheit using the formula:

$$F = \frac{9}{5} \times C + 32$$

where F is the temperature in Fahrenheit and C is the temperature in Celsius.

3.1.1 Program Requirements

- Your program should take a temperature in Celsius as input.
- It should calculate and display the equivalent temperature in Fahrenheit.

3.1.2 Input

```
Enter temperature in Celsius: 20
```

3.1.3 Output

```
Temperature in Fahrenheit: 68.0
```

3.2 BMI Calculator

Write a Python program that calculates the Body Mass Index (BMI) of an individual based on their weight (in kilograms) and height (in meters). The BMI is calculated using the formula:

$$BMI = \frac{\text{mass}}{\text{height}^2}$$

3.2.1 Program Requirements

- Your program should take the mass and height as input from the user.
- It should calculate and display the BMI with two decimal places.

3.2.2 Input

```
Enter mass (kg): 70
Enter height (m): 1.75
```

3.2.3 Output

BMI: 22.86

3.3 Stopwatch Display

Write a Python program that takes an input of time in seconds and converts it into hours, minutes, and seconds.

3.3.1 Program Requirements

- Your program should take an integer input representing time in seconds.
- It should calculate and display the equivalent time in hours, minutes, and seconds.

3.3.2 Input

Enter time in seconds: 86400

3.3.3 Output

Equivalent time: 24 hours, 0 minutes, 0 seconds